

# 스마트 응급실 환자 관리 시스템

김박장홍

병상 데이터 확인  
(가용 병상 수)



202510858 홍혜민 202510792 김주연  
20251010844 장다슬 202510803 박민채

# 목차

|    |                                |                      |
|----|--------------------------------|----------------------|
| 01 | TOPIC BACKGROUND               | 주제 선정 배경             |
| 02 | PROJECT PURPOSE                | 프로젝트 목표              |
| 03 | CORE FUNCTION                  | 핵심 기능 구성             |
| 04 | FLOW CHART & OPERATION FLOW    | 시스템 순서도 & 동작 흐름      |
| 05 | DATA STRUCTURE AND CODE        | 자료구조 및 코드 구조 설명      |
| 06 | FEEDBACK REFLECTIONS           | 피드백 반영 및 수정 사항       |
| 07 | IMPLEMENTATION SCREEN & METHOD | 구현 화면 & C언어 기반 구현 방식 |
| 08 | SCHEDULE & CONCLUSION          | 일정 & 기대효과와 결론        |



# > 주제 선정 배경

MBC 뉴스

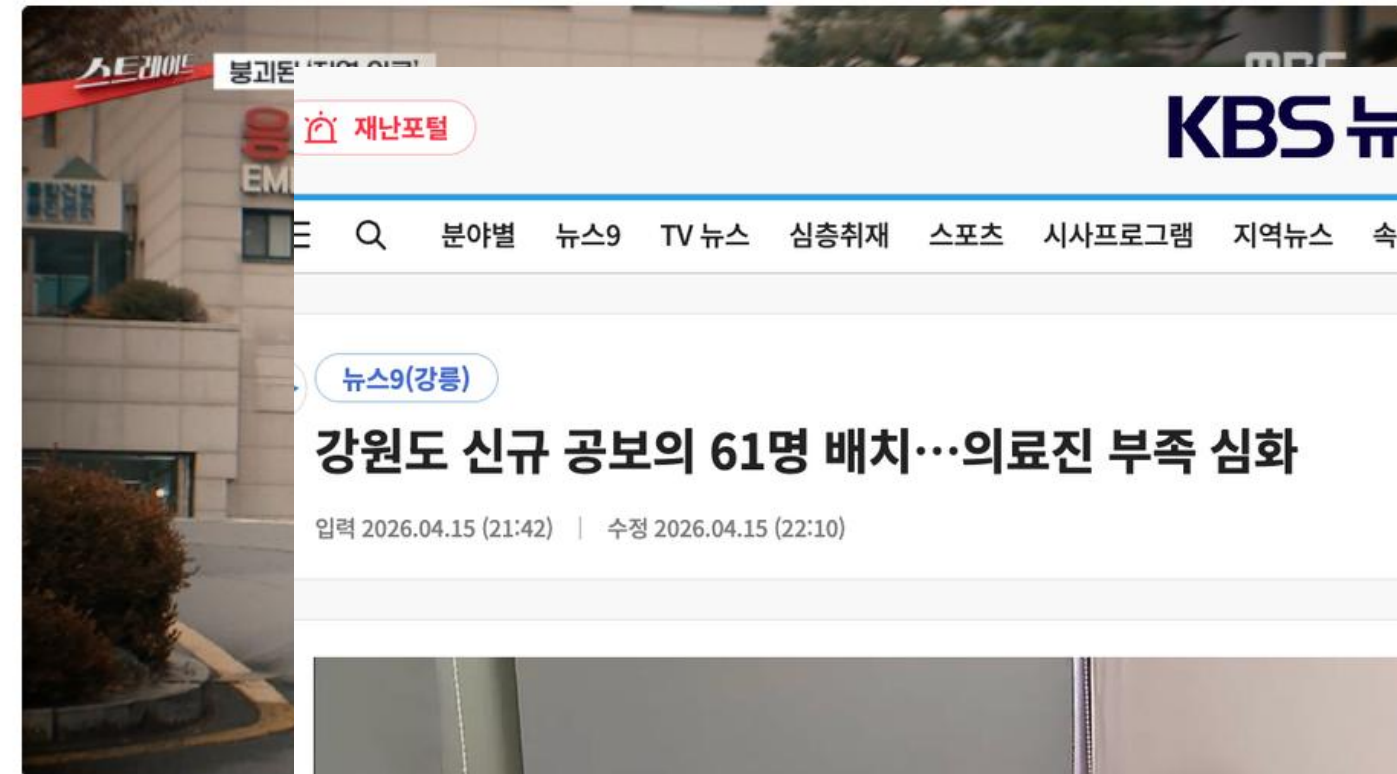
홈 | 뉴스데스크 | 뉴스투데이 | 심층 | 정치 | 사회 | 국제 | 경제 | 스포츠 | 엠빅뉴스 | 더보기 ▾

스트레이트 김태운

## [스트레이트] 끝나지 않은 '의료 대란'

입력 2025-12-21 21:13 | 수정 2025-12-21 22:22

3 가 댓글 공유



KBS 뉴스

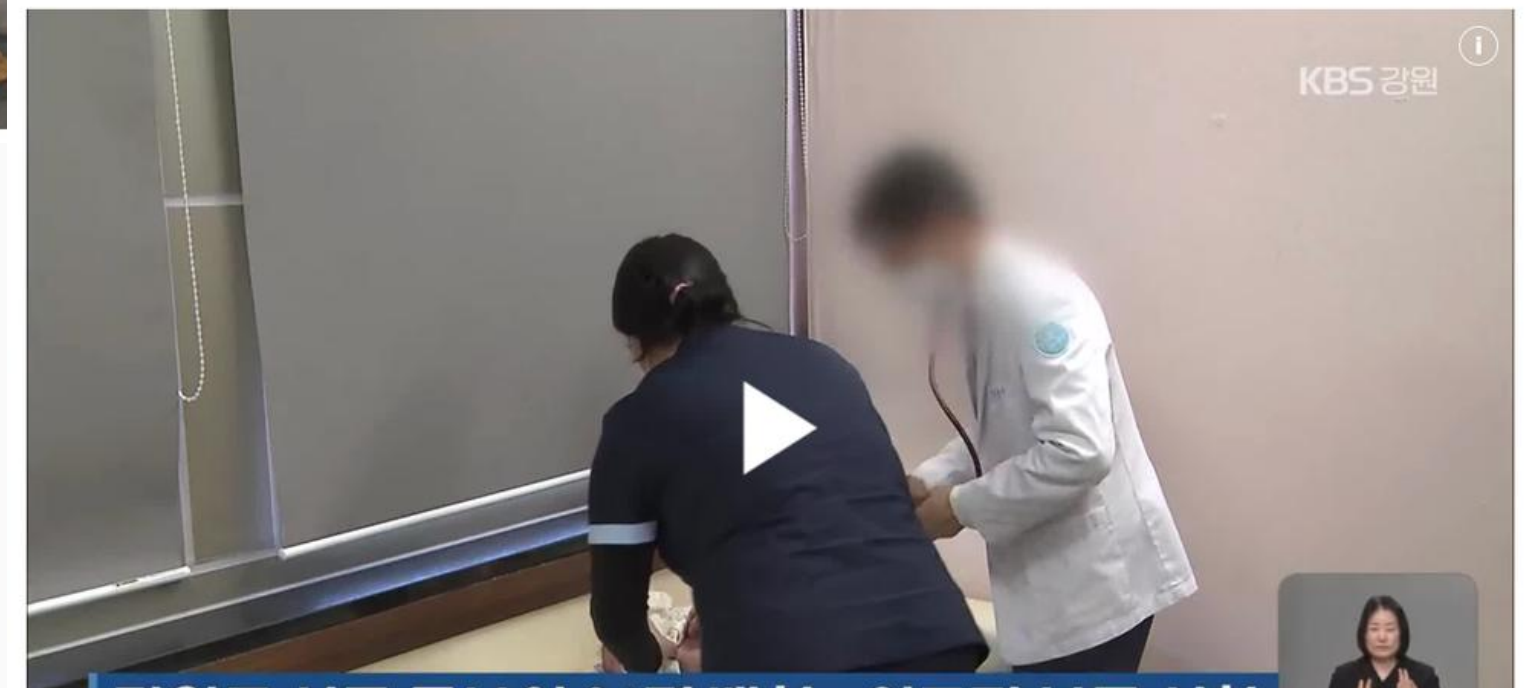
분야별 뉴스9 TV 뉴스 심층취재 스포츠 시사프로그램 지역뉴스 속보

뉴스9(강릉)

## 강원도 신규 공보의 61명 배치...의료진 부족 심화

입력 2026.04.15 (21:42) | 수정 2026.04.15 (22:10)

AI 요약



SBS NEWS 임신부 응급실 뱅뱅이...쌍둥이 1명 숨지고 1명 뇌손상

사회

## 임신부 응급실 뱅뱅이...쌍둥이 1명 숨지고 1명 뇌손상

유영규 기자 작성 2026.04.07 11:27 수정 2026.04.07 15:39 조회 4,574



▲ 자료사진

대구에서 조산 증세를 보인 쌍둥이 임신부가 이송 병원을 찾지 못해 4시간가량을 헤매다 끝내 아이 한 명을 잃고 다른 한 명도 중태에 빠진 사실이 뒤늦게 알려졌습니다.

## ➤ 주제 선정 배경



### 병상 부족

가용 병상이 없으면 새 환자가  
바로 배정 되지 못하고  
대기 상태가 발생함



### 상태 변화

대기 중에도 환자의 응급도가  
바뀔 수 있어 우선순위  
재조정이 필요함



### 대기 정보 부족

환자와 보호자는 현재 대기  
위치와 배정 상황을 직관적으로  
파악하기 어려움

따라서 단순 접수 순서가 아니라 중증도와 세부 중증도를 반영도를 반영하고 병상 변동에 따라  
대기 환자를 자동 재배정하는 구조가 필요함





## ➤ 프로젝트 목표

응급실 환자의 KTAS 중증도와 세부 중증도를 반영하여 우선순위를 관리하고 병상 상황에 따라 수동 배정 및 자동 재배정을 수행 할 수 있는 응급실 환자 관리 시스템을 구현한다

## > 시스템 핵심 기능 구성



### 신규 환자 접수

환자 ID, 이름, KTAS, 세부 중증도를 입력하고 heap, hash에 등록



### 수동 병상 배정

병상 여유가 있으면 Max Heap에서 최우선 환자를 추출해 배정



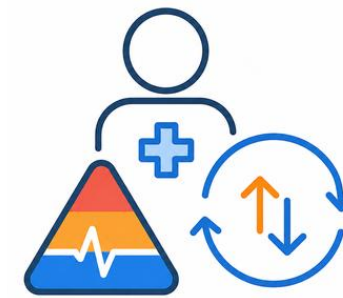
### 환자 대기 조회

Hash Table로 환자를 검색하고 현재 대기 상황을 안내



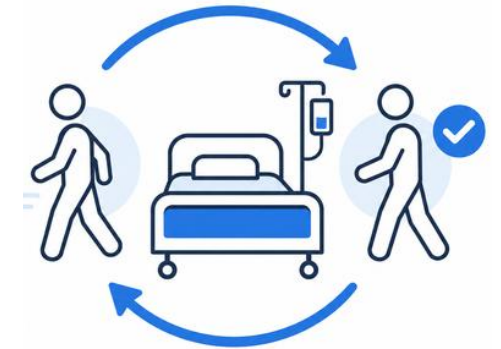
### 접수 취소

대기 중인 환자의 상태를 CANCELLED로 변경하고 heap 상태도 동기화



### 응급도 변경

이미 등록된 환자의 KTAS와 세부 중증도를 수정한 뒤 힙 재정렬



### 퇴원 후 자동 재배정

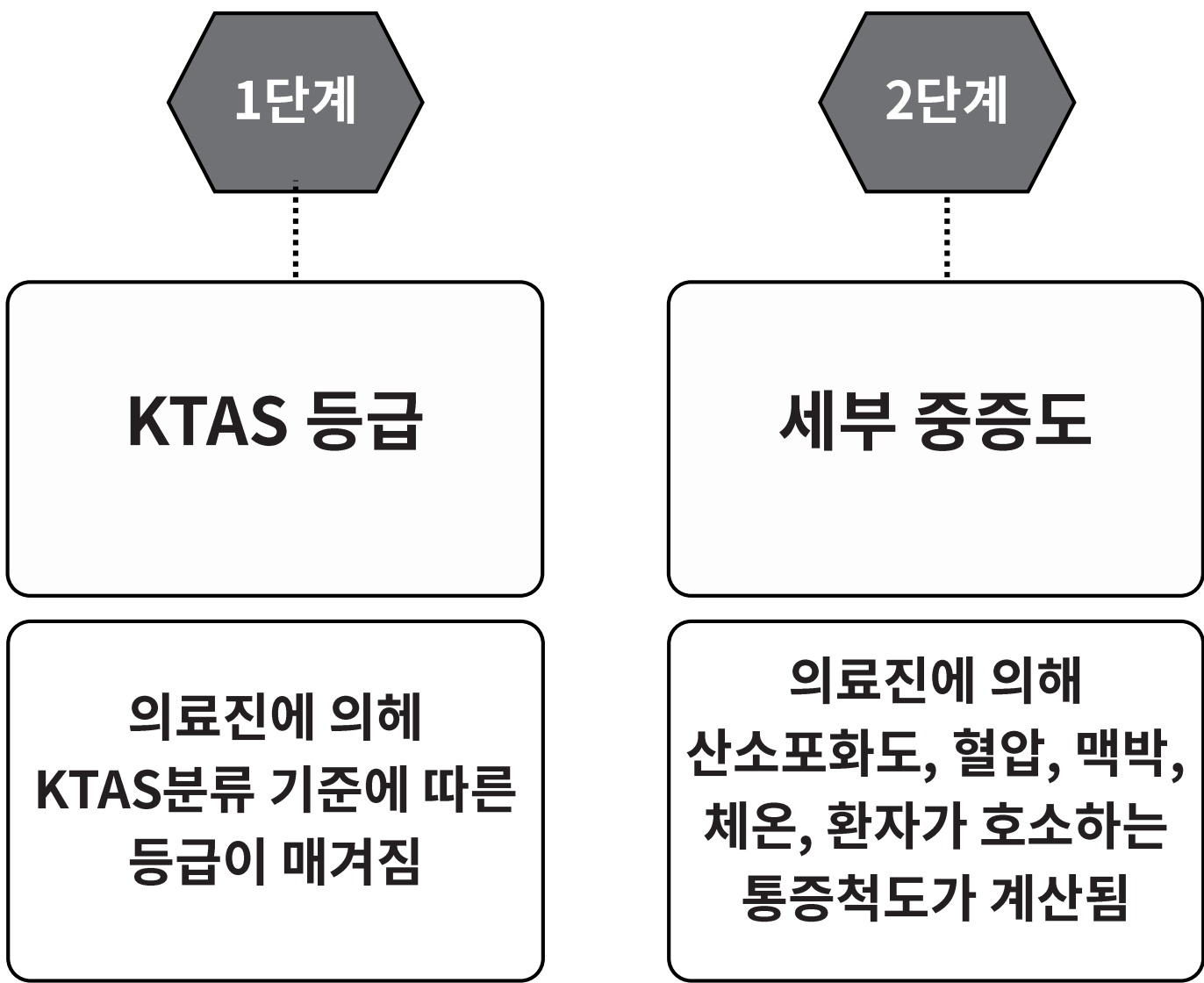
퇴원으로 자리가 생길 시 다음 최우선 대기 환자를 자동 배정



> 시스템 순서도 & 동작 흐름

우선순위 판단 기준

수정 반영 포인트: 같은 KTAS 등급 안에서도 세부 중증도를 추가로 반영함

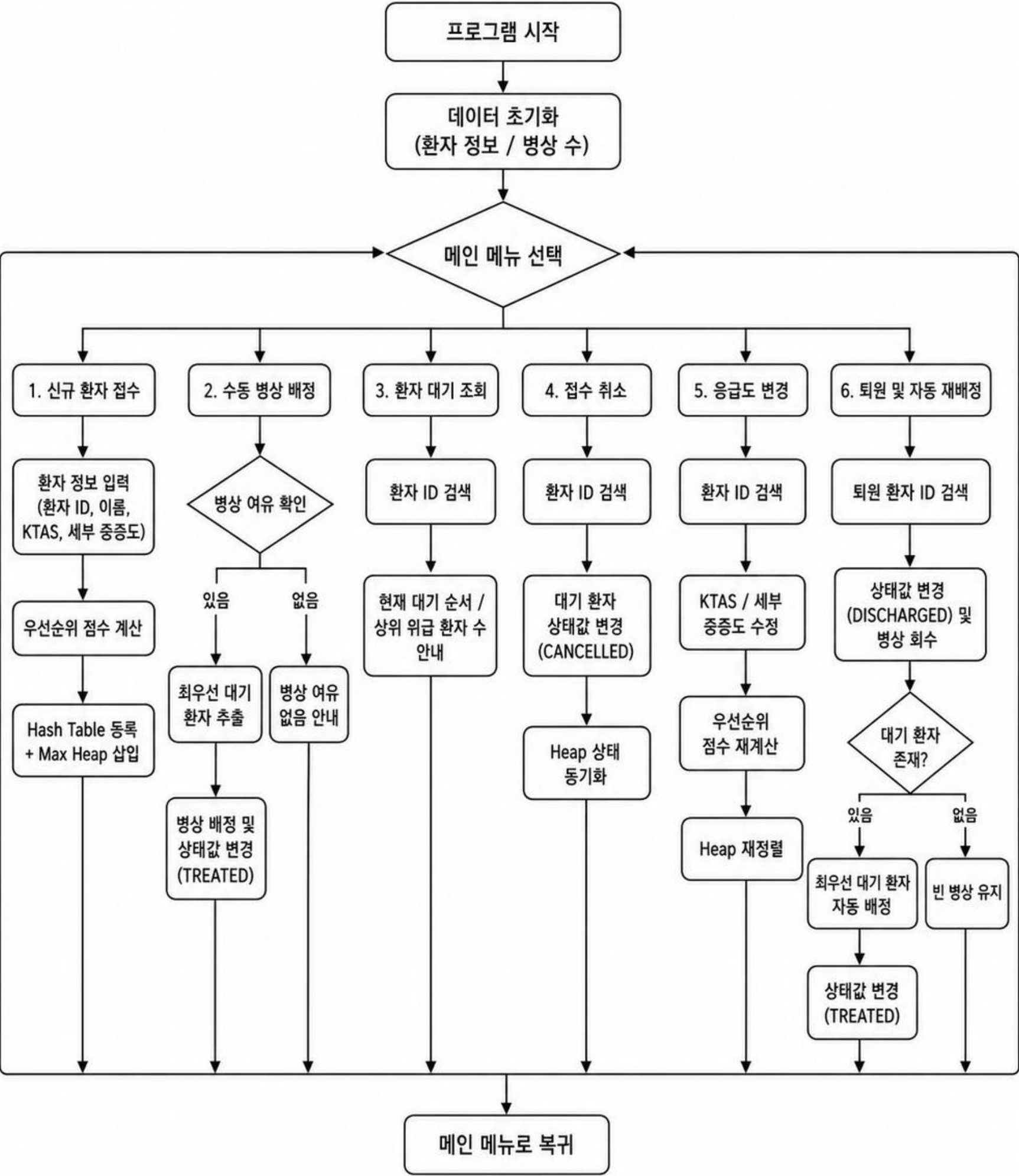


KTAS 분류 기준

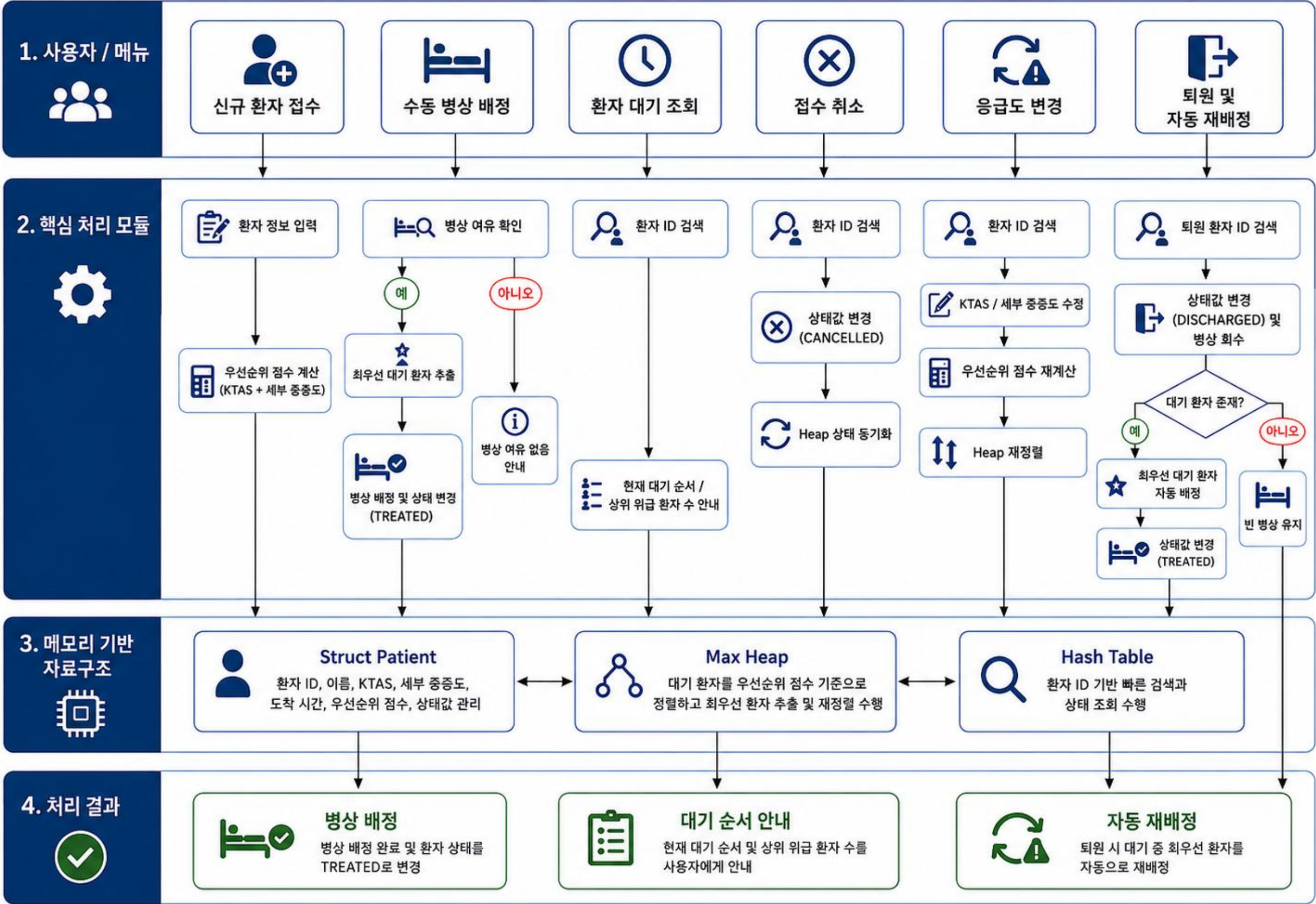
| 단계     |     | 단계별 정의                                                          | 대표적인 증상                          | 진료우선순위 |
|--------|-----|-----------------------------------------------------------------|----------------------------------|--------|
| KTAS 1 | 소생  | 즉각적인 처치가 필요하며 생명이나 사지를 위협하는(또는 악화 가능성이 높은) 상태                   | 심장마비, 무호흡, 음주와 관련되지 않은 무의식       | 최우선순위  |
| KTAS 2 | 긴급  | 생명 혹은 사지, 신체기능에 잠재적인 위험이 있으며 이에 대한 빠른 치료가 필요한 경우                | 심근경색, 뇌출혈, 뇌경색                   | 2순위    |
| KTAS 3 | 응급  | 치료가 필요한 상태로 진행할 수도 있는 잠재적 가능성을 고려해야 하는 경우                       | 호흡곤란(산소포화도 90%이상), 출혈을 동반한 설사    | 3순위    |
| KTAS 4 | 준응급 | 환자의 나이, 통증이나 악화/합병증에 대한 가능성을 고려할 때 1~2시간 안에 처치나 재평가를 시행하면 되는 상태 | 38도 이상의 발열을 동반한 장염, 복통을 동반한 요로감염 | 4순위    |
| KTAS 5 | 비응급 | 긴급하지만 응급은 아닌 상태, 만성적인 문제로 인한 것이거나, 악화의 가능성이 낮은 상태               | 감기, 장염, 설사, 열상(상처)               | 5순위    |



# 시스템 순서도 & 동작 흐름



# > 시스템 순서도 & 동작 흐름





# > 시스템 순서도 & 동작 흐름



01

## 1) 시스템 초기화

프로그램 실행 시 **heap**과 **hash table**을 초기화합니다.



02

## 2) 환자 접수

환자 ID 이름 **KTAS**  
세부 중증도를 입력받고  
우선순위 점수를 계산하여  
**heap**과 **hash**에 저장합니다.



03

## 3) 수동 배정

병상 여유가 있을 경우  
**heap**에서 최우선 환자를 선택하여 병상에 배정합니다.



04

## 4) 조회 / 취소

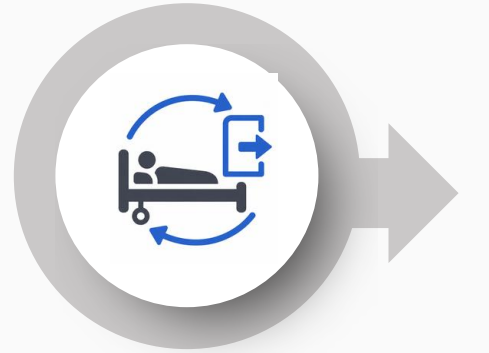
해시 검색을 통해 특정 환자의 상태를 빠르게 확인하고 대기 중인 환자에 대해서는 접수 취소 처리를 수행합니다



05

## 5) 응급도 변경

이미 등록된 환자의 **KTAS**와 세부 중증도를 수정하고 변경된 점수를 반영하여 힙을 재정렬합니다.



06

## 6) 퇴원 / 자동 재배정

퇴원으로 병상이 비면 다음 최우선 대기 환자를 자동으로 병상에 재배정합니다.

## ➤ 자료구조 및 코드 구조

### ✔ Struct Patient

---

환자 ID 이름 KTAS 세부 중증도 도착 시간  
우선순위 점수 상태값 등  
환자 정보를 하나의 구조체로 통합 관리

### ✔ Max Heap

---

중증도가 높은 환자를  
우선적으로 처리할 수 있도록  
우선순위가 가장 높은 환자를  
빠르게 추출하고 재정렬하는 데 사용

### ✔ Hash Table

---

환자 ID를 기준으로  
특정 환자 정보를 빠르게 검색하고  
조회 취소 상태 변경 시 원본 데이터에  
효율적으로 접근하는 데 사용

# > 자료구조 및 코드 구조

Common.h

환자 정보와 상대값 정의

heap.h / heap.c

우선순위 삽입 추출 재정렬

hash.h / hash.c

환자 ID 기반 검색

sharing.h / sharing.c

대기 조회 접수 취소

main.c

메뉴 제어와 전체 흐름 연결



## ➤ 자료구조 및 코드 구조

# 핵심 코드 화면

## <헤더파일>

common.h - 환자 정보 및 병상 데이터 구조체 정의

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
1  #pragma once
2  #ifndef SHARING_H
3  #define SHARING_H
4
5  #include "common.h" //보호자가 실시간 대기 상황 및 예상 순위 확인
6  int cancelPatient(int patient_id); // 환자 대기 접수 취소 처리
7
8  #endif
```

heap.h - 우선순위 정렬기능

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
1  #pragma once
2  #ifndef HEAP_H
3  #define HEAP_H
4
5  #include "common.h" // Patient 구조체를 사용하기 위해 인클루드
6
7  // Max Heap 전역 변수 선언 (extern을 써서 heap.c에 있는 변수를 공유)
8  extern Patient heap[];
9  extern int heapSize;
10
11  // 함수 선언
12  void initHeap();
13  void insertHeap(Patient p);
14  Patient extractMax();
15  void applyAging();
16  int updatePatientSeverity(int patient_id, KtasLevel new_ktas, int new_detail_score);
17
18  #endif
```

## > 자료구조 및 코드 구조

# 핵심 코드 화면

## <헤더파일>

### sharing.h - 실시간 대기 순서 공유 기능

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
1  #pragma once
2  #ifndef SHARING_H
3  #define SHARING_H
4
5  #include "common.h" //보호자가 실시간 대기 상황 및 예상 순위 확인
6  int cancelPatient(int patient_id); // 환자 대기 접수 취소 처리
7
8  #endif
```

### hash.h - 대기 순서 조회 상황에서 쓸 환자 정보 검색 기능

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위) insertHash
1  #pragma once
2  #ifndef HASH_H
3  #define HASH_H
4
5  #include "common.h"
6
7  // 함수 선언
8  int hashFunction(int id);
9  void insertHash(Patient p);
10 Patient* searchHash(int id); // 환자 데이터의 주소(포인터)를 반환해서 실시간 상태 변경이 가능하도록 함
11
12 #endif
```

## ➤ 자료구조 및 코드 구조

# 핵심 코드 화면 <소스파일>

main.c - 프로그램 메인 루프, 메뉴제어

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharin
datastructure_KPJH  (전역 범위)
1  #define _CRT_SECURE_NO_WARNINGS
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5  #include "heap.h"
6  #include "hash.h"
7  #include "sharing.h"
8
9  int main() {
10     int choice, id;
11     Patient new_patient;
12
13     // 병상 관리 변수 세팅
14     const int MAX_BEDS = 100;
15     int available_beds = MAX_BEDS;
16
17     initHeap();
18     initHash(); // 해시 테이블 초기화
19
20     while (1) {
21         printf("Wn=====Wn");
22         printf("   스마트 응급실 환자 관리 시스템Wn");
23         printf("=====Wn");
24         printf("1. 신규 환자 접수Wn");
25         printf("2. 다음 환자 진료 및 병상 배정Wn");
26         printf("3. 환자/보호자용 대기 순서 조회Wn");
27         printf("4. 접수 취소Wn");
28         printf("5. 환자 응급도 변경Wn");
29         printf("6. 환자 퇴원 처리Wn");
30         printf("메뉴 선택: ");
31         scanf("%d", &choice);
32
33         switch (choice) {
34             case 1:
35                 printf("Wn[신규 환자 접수]Wn");
36                 printf("환자 ID(생년월일): ");
37                 scanf("%d", &new_patient.id);
38                 printf("이름: ");
39                 scanf("%s", new_patient.name);
```

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH  (전역 범위)  main
40
41     // KTAS 등급 입력
42     printf("KTAS 등급 (1=최위급 ~ 5=비응급): ");
43     int ktas_input;
44     while (1) {
45         scanf("%d", &ktas_input);
46         if (ktas_input >= 1 && ktas_input <= 5) {
47             break; // 정상 범위 입력 시 루프 탈출
48         }
49         printf(">> 입력 오류!! KTAS 등급은 1에서 5 사이의 숫자만 가능합니다.Wn");
50         printf(">> KTAS 등급 재입력 (1-5): ");
51     }
52
53     // 세부 중증도 입력 및 예외 처리 (1-10)
54     printf("세부 중증도 (1-10점): ");
55     while (1) {
56         scanf("%d", &new_patient.detail_score);
57         if (new_patient.detail_score >= 1 && new_patient.detail_score <= 10) {
58             break; // 정상 범위 입력 시 루프 탈출
59         }
60         printf(">> [입력 오류] 세부 중증도는 1에서 10 사이의 숫자만 가능합니다.Wn");
61         printf(">> 세부 중증도 재입력 (1-10): ");
62     }
63
64     // 1단계가 점수가 높도록 역산 (1단계->5점, 5단계->1점)
65     new_patient.ktas = (KtasLevel)(6 - ktas_input);
66     new_patient.arrival_time = 0; // 실제 구현 시 time() 함수 사용 가능
67     new_patient.status = WAITING;
68     // 복합 우선순위 점수 계산 (중증도 우선) ***중증도 세분화로 수정***
69     new_patient.priority_score = (new_patient.ktas * 10000) + (new_patient.detail_score * 100);
70
71     insertHash(new_patient); // 해시에 0(1) 탐색용으로 저장
72     insertHeap(new_patient); // 힙에 위급도 순서대로 삽입
73     printf(">> %s 환자 접수 완료! (위급도 점수: %d)Wn", new_patient.name, new_patient.priority_score);
74     break;
75
76 case 2:
77     printf("Wn[수동 병상 배정]Wn");
78     // 병상 점유가 있는지 먼저 체크 (오스 바지)
```



## ➤ 자료구조 및 코드 구조

# 핵심 코드 화면 <소스파일>

main.c - 프로그램 메인 루프, 메뉴제어

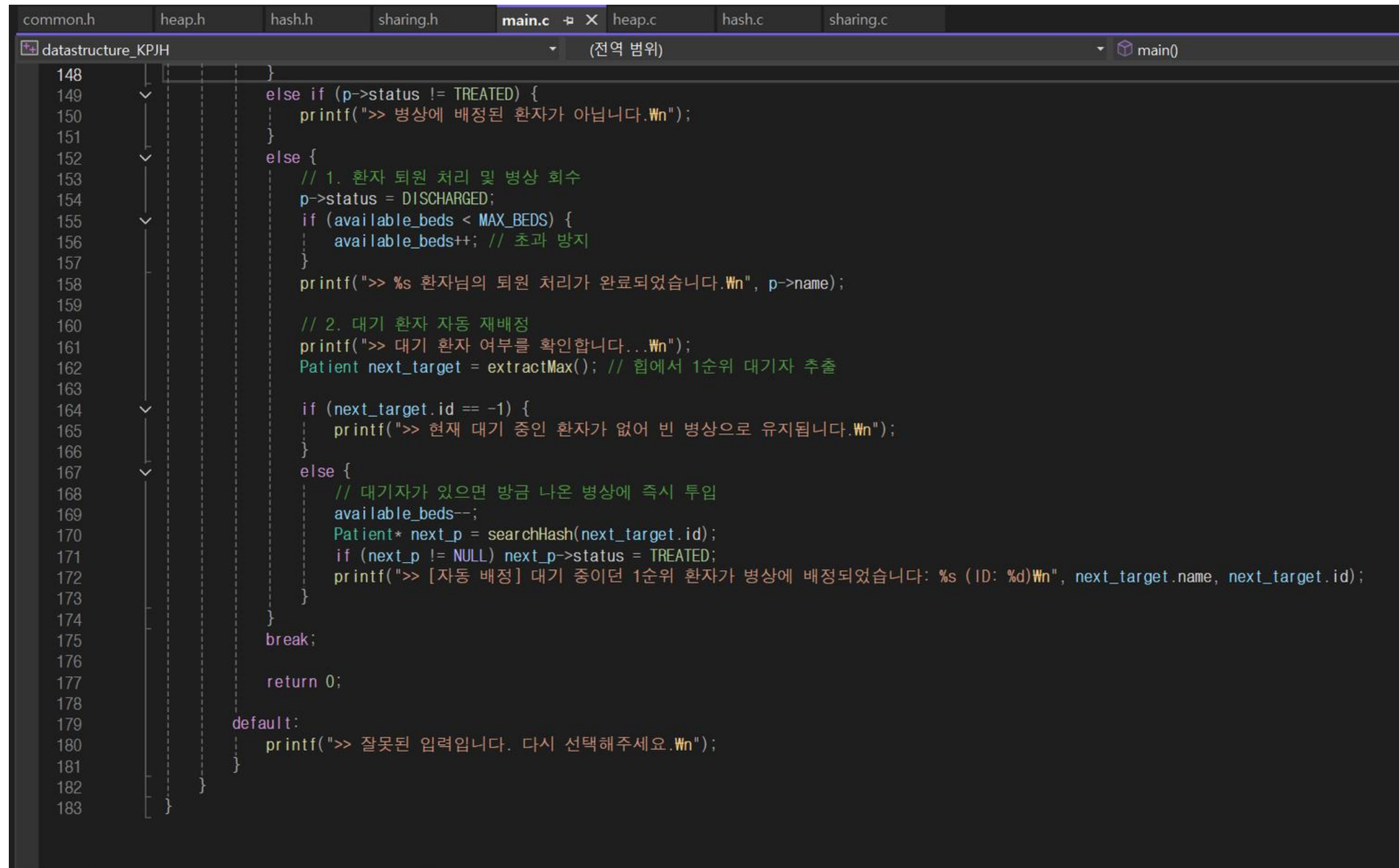
```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
76 case 2:
77     printf("Wn[수동 병상 배정]Wn");
78     // 병상 여유가 있는지 먼저 체크 (음수 방지)
79     if (available_beds <= 0) {
80         printf(">> 가용 병상이 부족합니다! 퇴원 처리가 먼저 필요합니다.Wn");
81     }
82     else {
83         Patient target = extractMax();
84         if (target.id == -1) {
85             printf(">> 대기 중인 환자가 없습니다.Wn");
86         }
87         else {
88             available_beds--; // 병상 수 차감
89             printf(">> 병상 배정 완료: %s (ID: %d)Wn", target.name, target.id);
90             Patient* p = searchHash(target.id);
91             if (p != NULL) p->status = TREATED;
92         }
93     }
94     break;
95
96 case 3:
97     printf("Wn[실시간 대기 순서 조회]Wn");
98     printf("조회할 환자 ID 입력: ");
99     scanf("%d", &id);
100     displayPatientOrder(id);
101     break;
102
103 case 4:
104     printf("Wn[대기 접수 취소]Wn");
105     printf("취소할 환자 ID 입력: ");
106     scanf("%d", &id);
107     cancelPatient(id);
108     break;
109
110 case 5:
111     printf("Wn[환자 응급도 변경]Wn");
112     printf("응급도를 수정할 환자 ID 입력: ");
113     scanf("%d", &id);
```

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
115     printf("새로운 KTAS 등급 (1~5): ");
116     int new_ktas_input;
117     scanf("%d", &new_ktas_input);
118     KtasLevel new_ktas = (KtasLevel)(6 - new_ktas_input);
119
120     printf("새로운 세부 중증도 (1~10점): ");
121     int new_detail_score;
122     scanf("%d", &new_detail_score);
123
124     // 힘 재조정 시도
125     if (updatePatientSeverity(id, new_ktas, new_detail_score)) {
126         // 힘이 업데이트되었다면 해시 테이블 원본 데이터도 동기화!
127         Patient* target_hash = searchHash(id);
128         if (target_hash != NULL) {
129             target_hash->ktas = new_ktas;
130             target_hash->detail_score = new_detail_score;
131             target_hash->priority_score = (new_ktas * 10000) + (new_detail_score * 100);
132         }
133         printf(">> 상태가 업데이트되어 대기열 우선순위가 재조정되었습니다.Wn");
134     }
135     else {
136         printf(">> 대기 중인 환자를 찾을 수 없습니다.Wn");
137     }
138     break;
139
140 case 6:
141     printf("Wn[환자 퇴원 및 병상 정리]Wn");
142     printf("퇴원할 환자 ID 입력: ");
143     scanf("%d", &id);
144
145     Patient* p = searchHash(id);
146     if (p == NULL) {
147         printf(">> 등록되지 않은 환자입니다.Wn");
148     }
149     else if (p->status != TREATED) {
150         printf(">> 병상에 배정된 환자가 아닙니다.Wn");
151     }
152     else {
153         // 1. 환자 퇴원 처리 및 병상 환수
```

## ➤ 자료구조 및 코드 구조

# 핵심 코드 화면 <소스파일>

main.c - 프로그램 메인 루프, 메뉴제어



```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH  (전역 범위)  main()

148 }
149 else if (p->status != TREATED) {
150     printf(">> 병상에 배정된 환자가 아닙니다.\n");
151 }
152 else {
153     // 1. 환자 퇴원 처리 및 병상 회수
154     p->status = DISCHARGED;
155     if (available_beds < MAX_BEDS) {
156         available_beds++; // 초과 방지
157     }
158     printf(">> %s 고객님의 퇴원 처리가 완료되었습니다.\n", p->name);
159
160     // 2. 대기 환자 자동 재배정
161     printf(">> 대기 환자 여부를 확인합니다...\n");
162     Patient next_target = extractMax(); // 힙에서 1순위 대기자 추출
163
164     if (next_target.id == -1) {
165         printf(">> 현재 대기 중인 환자가 없어 빈 병상으로 유지됩니다.\n");
166     }
167     else {
168         // 대기자가 있으면 방금 나온 병상에 즉시 투입
169         available_beds--;
170         Patient* next_p = searchHash(next_target.id);
171         if (next_p != NULL) next_p->status = TREATED;
172         printf(">> [자동 배정] 대기 중이던 1순위 환자가 병상에 배정되었습니다: %s (ID: %d)\n", next_target.name, next_target.id);
173     }
174 }
175 break;
176
177 return 0;
178
179 default:
180     printf(">> 잘못된 입력입니다. 다시 선택해주세요.\n");
181 }
182 }
183 }
```



## ➤ 자료구조 및 코드 구조

# 핵심 코드 화면 <소스파일>

## 2. heap.c - 중증도 점수 활용 우선순위 정렬 구현

(신규 환자 접수, 1순위 환자 추출, 응급도 변화시 트리 재정렬)

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
37
38     if (heap[left].priority_score > heap[largest].priority_score)
39         largest = left;
40     if (right < heapSize && heap[right].priority_score > heap[largest].priority_score)
41         largest = right;
42
43     if (largest != current) {
44         swap(&heap[current], &heap[largest]);
45         current = largest;
46     }
47     else {
48         break;
49     }
50 }
51
52 void insertHeap(Patient p) {
53     if (heapSize >= MAX_HEAP) {
54         printf("대기열이 가득 찼습니다!\n");
55         return;
56     }
57     heap[heapSize] = p;
58     reheapifyUp(heapSize); // 분리한 함수 호출
59     heapSize++;
60 }
61
62 Patient extractMax() {
63     Patient empty_patient = { -1, "", 0, 0, 0, 0, CANCELLED };
64     while (heapSize > 0) {
65         Patient maxPatient = heap[0];
66         heap[0] = heap[heapSize - 1];
67         heapSize--;
68         reheapifyDown(0); // 분리한 함수 호출
69
70         if (maxPatient.status != CANCELLED) {
71             return maxPatient;
72         }
73     }
74     return empty_patient;
75 }
```

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
1  #include "heap.h"
2  #include <stdio.h>
3
4  #define MAX_HEAP 100
5
6  Patient heap[MAX_HEAP];
7  int heapSize = 0;
8
9  void initHeap() { heapSize = 0; }
10
11 void swap(Patient* a, Patient* b) {
12     Patient temp = *a;
13     *a = *b;
14     *b = temp;
15 }
16
17 // Re-heapify Up (위로 끌어올리기) 함수 분리
18 void reheapifyUp(int current) {
19     while (current > 0) {
20         int parent = (current - 1) / 2;
21         if (heap[current].priority_score > heap[parent].priority_score) {
22             swap(&heap[current], &heap[parent]);
23             current = parent;
24         }
25         else {
26             break;
27         }
28     }
29 }
30
31 // Re-heapify Down (아래로 내리기) 함수 분리
32 void reheapifyDown(int current) {
33     while (current * 2 + 1 < heapSize) {
34         int left = current * 2 + 1;
35         int right = current * 2 + 2;
36         int largest = current;
37
38         if (heap[left].priority_score > heap[largest].priority_score)
39             largest = left;
40         if (right < heapSize && heap[right].priority_score > heap[largest].priority_score)
41             largest = right;
42
43         if (largest != current) {
44             swap(&heap[current], &heap[largest]);
45             current = largest;
46         }
47     }
48 }
```

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
73
74 }
75 return empty_patient;
76 }
77
78 // 접수된 환자의 응급도 변화 시 특정 환자의 위치를 재조정하는 함수
79 int updatePatientSeverity(int patient_id, KtasLevel new_ktas, int new_detail_score) {
80     int target_idx = -1;
81
82     // 1. 힙 배열에서 대상 환자의 인덱스 찾기
83     for (int i = 0; i < heapSize; i++) {
84         if (heap[i].id == patient_id && heap[i].status == WAITING) {
85             target_idx = i;
86             break;
87         }
88     }
89     if (target_idx == -1) return 0; // 대기 중인 환자를 못 찾음
90
91     int old_score = heap[target_idx].priority_score;
92     int new_score = (new_ktas * 10000) + (new_detail_score * 100);
93
94     // 2. 점수 업데이트
95     heap[target_idx].ktas = new_ktas;
96     heap[target_idx].detail_score = new_detail_score;
97     heap[target_idx].priority_score = new_score;
98
99     // 3. 점수가 올랐으면 Up, 내렸으면 Down하여 힙 구조 복구
100     if (new_score > old_score) {
101         reheapifyUp(target_idx);
102     }
103     else if (new_score < old_score) {
104         reheapifyDown(target_idx);
105     }
106     return 1;
107 }
```



## ➤ 자료구조 및 코드 구조

# 핵심 코드 화면 <소스파일>

## 4. sharing.c - 실시간 대기 순서 공유, 접수취소 기능

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c
datastructure_KPJH (전역 범위)
1  #include "hash.h"
2  #include <stdio.h>
3  #include <string.h>
4
5  #define TABLE_SIZE 100
6
7  // 해시 테이블 배열 (포인터 배열 아님, 직접 저장)
8  Patient hashTable[TABLE_SIZE];
9
10 void initHash() {
11     for (int i = 0; i < TABLE_SIZE; i++) {
12         hashTable[i].id = -1;
13     }
14 }
15
16 int hashFunction(int id) {
17     return id % TABLE_SIZE;
18 }
19
20 void insertHash(Patient p) {
21     int index = hashFunction(p.id);
22
23     // 빈 자리를 찾을 때까지 이동
24     while (hashTable[index].id != -1) {
25         index = (index + 1) % TABLE_SIZE;
26     }
27     hashTable[index] = p;
28 }
29
30 Patient* searchHash(int id) {
31     int index = hashFunction(id);
32     int startIndex = index;
33
34     while (hashTable[index].id != -1) {
35         if (hashTable[index].id == id) {
36             return &hashTable[index]; // 주소 반환
37         }
38         index = (index + 1) % TABLE_SIZE;
39         // 한 바퀴 다 돌았으면 종료
40         if (index == startIndex) break;
41     }
42     return NULL; // 못 찾았을 경우
```

```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위)
1  #include "sharing.h"
2  #include "hash.h"
3  #include "heap.h"
4  #include <stdio.h>
5
6  void displayPatientOrder(int patient_id) {
7      Patient* target = searchHash(patient_id);
8      if (target == NULL) {
9          printf(">> 등록되지 않은 환자 번호입니다.\n");
10         return;
11     }
12     if (target->status == CANCELLED) {
13         printf(">> 이미 접수가 취소된 환자입니다.\n");
14         return;
15     }
16     if (target->status == TREATED) {
17         printf(">> 병상 배정된 환자입니다.\n");
18         return;
19     }
20
21     int higher_ktas_count = 0;
22     int same_ktas_ahead_count = 0;
23
24     // 힙 배열을 순회하며 앞순서 대기자 분석
25     for (int i = 0; i < heapSize; i++) {
26         if (heap[i].status != WAITING) continue;
27         if (heap[i].id == patient_id) continue;
28
29         if (heap[i].ktas > target->ktas) {
30             higher_ktas_count++;
31         }
32         else if (heap[i].ktas == target->ktas && heap[i].priority_score > target->priority_score) {
33             same_ktas_ahead_count++;
34         }
35     }
36
37     printf("\n=====");
38     printf("실시간 대기 현황 및 안내 (보호자용)\n");
39     printf("=====");
40     printf("환자명: %s (KTAS 등급: %d)\n", target->name, (6 - target->ktas));
41     printf("-----");
42     printf("나보다 위급한 중증 환자 수: %d명\n", higher_ktas_count);
43     printf("나보다 위급한 중증 환자 수: %d명\n", same_ktas_ahead_count);
44 }
```

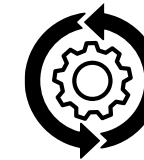
## > 피드백 반영 및 수정 사항

```
common.h  heap.h  hash.h  sharing.h  main.c*  heap.c  hash.c  sharing.c
datastructure_KPJH  (전역 범위)
106      scanf("%d", &id);
107      cancelPatient(id);
108      break;
109
110  case 5:
111      printf("환자 응급도 변경\n");
112      printf("응급도를 수정할 환자 ID 입력: ");
113      scanf("%d", &id);
114
115      printf("새로운 KTAS 등급 (1~5): ");
116      int new_ktas_input;
117      scanf("%d", &new_ktas_input);
118      KtasLevel new_ktas = (KtasLevel)(6 - new_ktas_input);
119
120      printf("새로운 세부 중증도 (1~10점): ");
121      int new_detail_score;
122      scanf("%d", &new_detail_score);
123
124      // 힘 재조정
125      if (updatePatientSeverity(id, new_ktas, new_detail_score)) {
126          // 힘이 업데이트 후 해시 테이블 원본 데이터도 동기화!
127          Patient* target_hash = searchHash(id);
128          if (target_hash != NULL) {
129              target_hash->ktas = new_ktas;
130              target_hash->detail_score = new_detail_score;
131              target_hash->priority_score = (new_ktas * 10000) + (new_detail_score * 100);
132          }
133          printf(">> 상태가 업데이트되어 대기열 우선순위가 재조정되었습니다.\n");
134      }
135      else {
136          printf(">> 대기 중인 환자를 찾을 수 없습니다.\n");
137      }
138      break;
139
140  case 6:
```



### 피드백

같은 등급 안에서도 중증도가 차이날텐데  
어떻게 판별할 것인가?



### 개선방안

세부 중증도 기능을 제작하여 같은 등급 내  
에서 더 위급한 환자를 구별

## > 피드백 반영 및 수정 사항

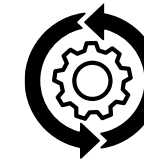
```
common.h  heap.h  hash.h  sharing.h  main.c*  heap.c  hash.c  sharing.c
datastructure_KPJH  (전역 범위)  main()

109
110     case 5:
111         printf("Wn[ 환자 응급도 변경 ]Wn");
112         printf("응급도를 수정할 환자 ID 입력: ");
113         scanf("%d", &id);
114
115         printf("새로운 KTAS 등급 (1~5): ");
116         int new_ktas_input;
117         scanf("%d", &new_ktas_input);
118         KtasLevel new_ktas = (KtasLevel)(6 - new_ktas_input);
119
120         printf("새로운 세부 중증도 (1~10점): ");
121         int new_detail_score;
122         scanf("%d", &new_detail_score);
123
124         // 힘 재조정
125         if (updatePatientSeverity(id, new_ktas, new_detail_score)) {
126             // 힘이 업데이트 후 해시 테이블 원본 데이터도 동기화!
127             Patient* target_hash = searchHash(id);
128             if (target_hash != NULL) {
129                 target_hash->ktas = new_ktas;
130                 target_hash->detail_score = new_detail_score;
131                 target_hash->priority_score = (new_ktas * 10000) + (new_detail_score * 100);
132             }
133             printf(">> 상태가 업데이트되어 대기열 우선순위가 재 조정되었습니다.Wn");
134         }
135         else {
136             printf(">> 대기 중인 환자를 찾을 수 없습니다.Wn");
137         }
```



### 피드백

이미 접수를 한 환자의 응급도  
수정 가능 여부



### 개선방안

응급도 수정이 가능하도록  
응급도 수정 (5번) 기능을 개발



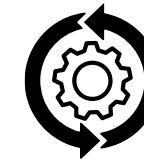
## ➤ 피드백 반영 및 수정 사항



### 피드백

환자퇴원부터 대기환자 재배정이  
이뤄지도록 코드를 수정하는 게  
좋을 거 같음

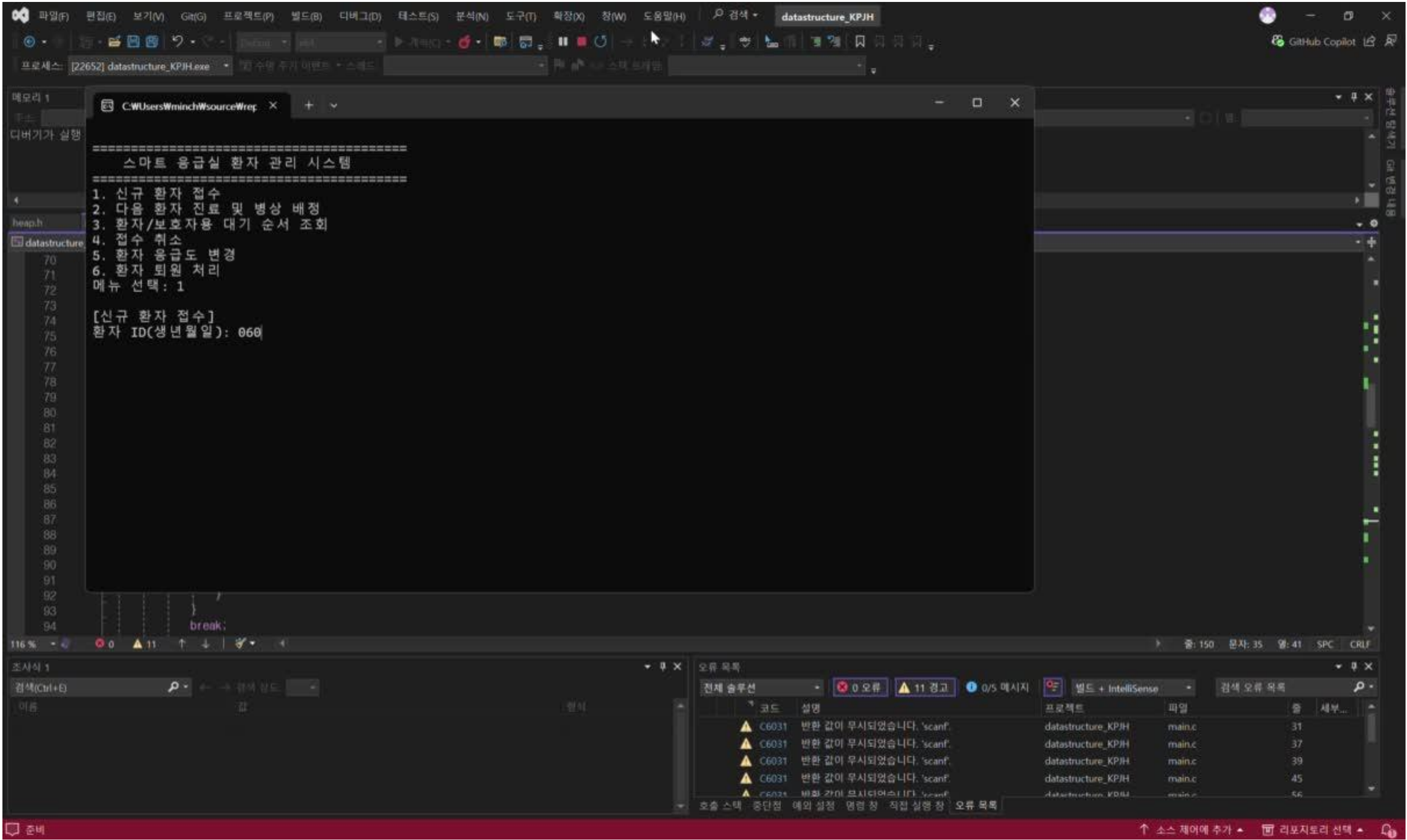
```
common.h  heap.h  hash.h  sharing.h  main.c  heap.c  hash.c  sharing.c
datastructure_KPJH (전역 범위) main()
140 case 6:
141     printf("환자 퇴원 및 병상 정리\n");
142     printf("퇴원할 환자 ID 입력: ");
143     scanf("%d", &id);
144
145     Patient* p = searchHash(id);
146     if (p == NULL) {
147         printf(">> 등록되지 않은 환자입니다.\n");
148     }
149     else if (p->status != TREATED) {
150         printf(">> 병상에 배정된 환자가 아닙니다.\n");
151     }
152     else {
153         // 1. 환자 퇴원 처리 및 빈병상 처리
154         p->status = DISCHARGED;
155         if (available_beds < MAX_BEDS) {
156             available_beds++; // 초과 방지
157         }
158         printf(">> %s 환자의 퇴원 처리가 완료되었습니다.\n", p->name);
159
160         // 2. 대기 환자 자동 재배정
161         printf(">> 대기 환자 여부를 확인합니다...\n");
162         Patient next_target = extractMax(); // 힙에서 1순위 대기자 추출
163
164         if (next_target.id == -1) {
165             printf(">> 현재 대기 중인 환자가 없어 빈 병상으로 유지됩니다.\n");
166         }
167         else {
168             // 대기자가 있으면 방금 나온 병상에 즉시 투입
169             available_beds--;
170             Patient* next_p = searchHash(next_target.id);
171             if (next_p != NULL) next_p->status = TREATED;
172             printf(">> [자동 배정] 대기 중이던 1순위 환자가 병상에 배정되었습니다: %s (ID: %d)\n", next_target.name, next_target.id);
173         }
174     }
175 }
```



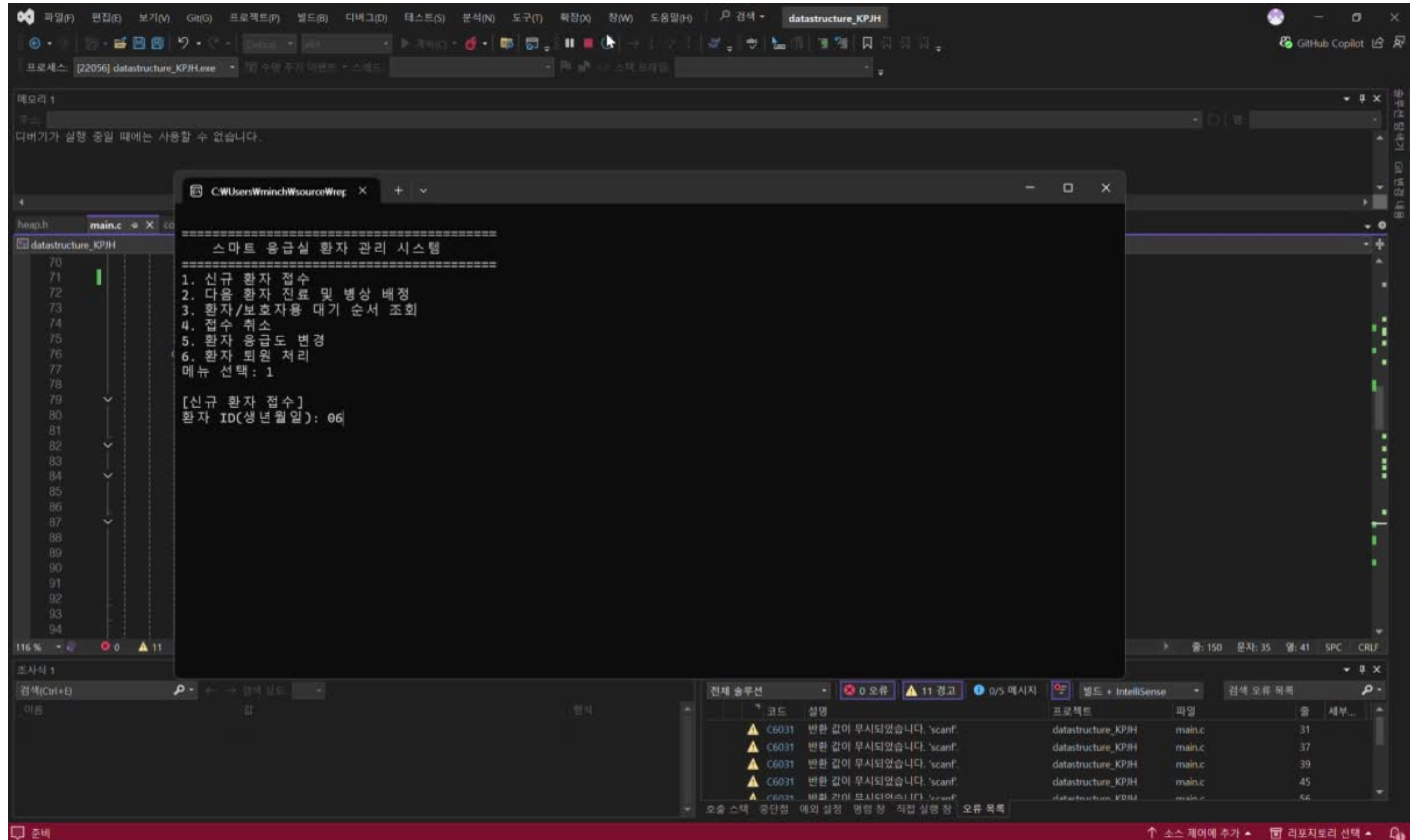
### 개선방안

환자 퇴원 시, 다음 대기환자가 병상으로 자  
동배정이 가능하도록 기능 수정

# 구현화면



# 구현화면





## > C언어 기반 구현 방식

환자 정보를 입력받아  
구조체 단위로 저장하고  
KTAS와 세부 중증도를  
반영해 우선순위 점수를  
계산하도록 구현



신규 환자 등록 및 정보 관리

우선순위 기반 환자 관리



Max Heap을 활용하여  
대기 환자를  
우선순위 순으로 관리하고  
최우선 환자를 빠르게  
추출할 수 있도록 구현

C언어

병상 여유가 있을 때  
수동 배정을 수행하고  
환자 상태를 변경하여  
병상 배정 과정을  
처리하도록 구현



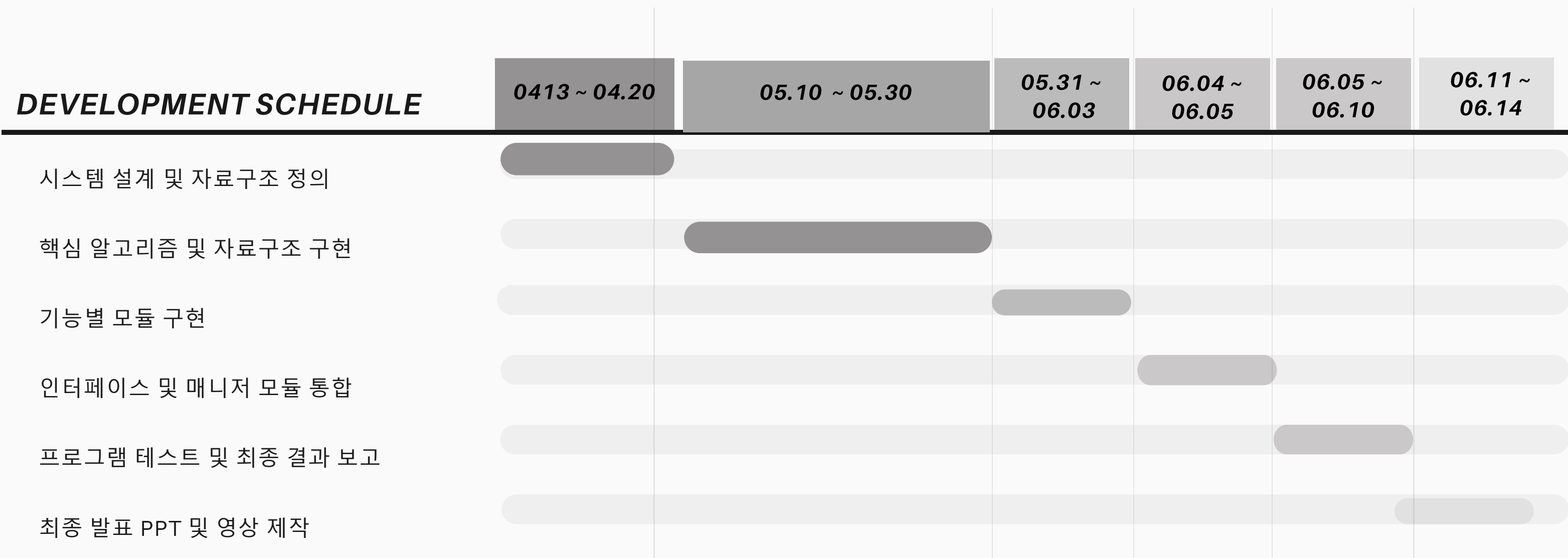
병상 배정 및 상태 변경

조회 변경 자동 재배정 기능



환자 ID 기반 조회 점수 취소  
응급도 변경  
퇴원 후 자동 재배정까지  
가능하도록 기능을 구현

# > 개발일정



## > 기대효과와 결론



### 최적의 의료 자원 배분

단순 선착순이 아닌 Max Heap 기반의  
중증도 우선순위 시스템을 통해  
골든타임 확보 및 응급실 효율 극대화



### 형평성 있는 대기 관리

중증 환자 우선 처리를 유지하면서도  
경증 환자의 기약 없는 대기 방지



### 신속하고 정확한 정보 처리

Hash Table을 통한 빠른 속도의 환자  
조회



### 실무 지향적 데이터 관리

$O(1)$ 를 통한 유연한  
명단 관리와 File I/O 연동으로  
실제 의료 현장에 적합한  
데이터 영속성 제공



# 경청해 주셔서 감사합니다.

김박장홍

